ContractForge: Verified Lean 4 Code from English Requirements

Mid-Submission Progress Report

Bavya Sri Kilari (2025701029), Akshith K (2024111004), Vivek R (2019112020)
Autonomous Software Engineering, Monsoon 2026, IIIT Hyderabad

6 November 2026 Repository: github.com/Ohshit-ak/AUTOSE-Leaniers

1 Where the project stands

ContractForge takes a software requirements specification (SRS) written in English and tries to produce Lean 4 code that is checked against it. We work in three layers. L0 turns each requirement into a Lean specification (`Spec : Contract Input Output`, a precondition and a postcondition). L1 groups the specifications into components, writes a contract for each, and proves that consecutive contracts fit together (PO1) and that their composition refines the specification (PO2). L2 writes an implementation per component and proves it meets its contract (PO3). A soundness theorem in our framework, `Contract.system_sound`, combines the three into end-to-end correctness. Lean decides at every layer whether the output is accepted, so nothing moves forward because a model said it was correct.

In short: L0 works and is frozen (version 10.1, about 90% of the gamma j requirements compile and most pass our extra checks). L1 runs end to end on the real SRS and proves 96 of 98 obligations per run, but the last obligation, and with it PO2, has not gone through yet. L2 has its first end-to-end results on a simpler path: 19 of 61 gamma j specifications and 7 of 14 video-search specifications are verified from specification to implementation, although many of those decompositions are shallow (Section 5). Everything runs on a local Qwen 2.5 7B model on the university GPU cluster.

The repository has 76 commits between 19 October and 6 November. Work happened on feature branches (`fix/l0-spec-quality`, `l0-v10`, `l1-graph-contracts`, `l1_runs`, `feat/l2-coding-agent`) that are merged into `main` through pull requests #1 and #2. `MEMORY.md` in the repository logs every version, decision and run in more detail than fits here.

2 Architecture and implementation

The Lean side (`framework/`) is small and uses Lean core only, without Mathlib, to keep builds fast. `Contract.lean` defines the `Contract` type, the three proof obligations and sequential composition; `Soundness.lean` and `Chain.lean` hold the soundness theorems for two and for n components; `Strings.lean` has decidable helpers for string formats such as `MM/YY`; and the tactic `discharge_po [A, B]` unfolds the named contracts and tries `simp_all`, `omega`, `grind` and `decide`.

The Python harness (`harness/`) has one entry point, `main.py`, with the stages parse, spec, check, graph, arch and code. The parser (`req_parser.py`) is rule-based and uses no model: the same XML always gives the same JSON, and every sentence it drops is stored with the rule that dropped it (219 dropped sentences on gamma j, mostly UI wording, vague goals and technology choices). All agents share one loop in `agents/base.py`: generate, check with Lean, repair. The repair step shows the model only the earliest layer of errors (syntax, then names, then types, then proofs), applies mechanical fixes first, suggests the closest real names when the model invents a function, and rejects a repair that weakens the specification. Locked context, such as an approved spec, is prepended by the harness, so a later agent cannot change it.

For the stage after compilation we built `validate.py` (we call it L0.5). It checks that `post` actually rejects a changed output, that runtime bounds found by the parser appear in the spec, and, for data-dictionary fields, that the spec accepts valid sample strings and rejects invalid ones generated from the SRS rules. It also runs a negative check: the model names one forbidden behaviour (for a login, a wrong password that still signs in), and Lean has to prove the spec rejects it. This check exists because one of our early login specs compiled and passed everything while letting every login succeed.

L1 is driven by a requirements graph (`req_graph.py`) that is built without a model. Operations are the L0 specs; entities come from a reviewed `Domain.lean` for the shop, the data dictionary, the glossary and a small synonym file. An edge goes from an operation that writes an entity to one that reads it. Connected groups become systems, large groups are split with Louvain clustering (fixed seed), cycles are merged, and the topological order gives the chain of components. The harness, not the model, writes every theorem statement in L1: the system spec is the conjunction of the L0 specs, the trace obligations say each contract assumes no more than its L0 spec and guarantees at least as much, and PO1/PO2 follow the chain. The model only writes two definitions per operation (`<C>_pre_<op>` and `<C>_post_<op>`), so it cannot make the goal easier. A `-no-graph` mode lets the model decompose the system itself

and serves as our baseline.

Models are called through one function, `call_llm`, which supports the local Qwen server, DeepSeek, Groq and Gemma. Runs are seeded, `-repeat N` produces several repetitions, and `evaluation/compare_runs.py` reports mean and range and only calls a difference "better" or "worse" when it is larger than the spread between repetitions. There are 128 unit tests on `main`, covering the parser, the checks, the graph, every L1 tool and the L2 coder.

3 Changes from the proposal

1. **Real SRS documents instead of a hand-made benchmark.** The proposal planned ArchVeriBench, 15–25 small hand-written systems. We use public SRS documents from the PURE dataset instead: the gamma j web store (75 entries) as the development set and a video-search SRS (21 entries) as a second document. This is a harder and more realistic input, and it meant much more time went into L0 than we planned. We keep a hand-written gold file (25 L0 specs for gamma j) for checking meaning.
2. **L1 contracts work on a message type.** We first tried contracts over a shared `World` record, but one `World` → `World` step cannot satisfy operations that conflict, such as login and logout. Contracts now act on a message type the harness generates from the L0 specs.
3. **The harness writes all L1 statements**, including the system spec, so the model cannot weaken the goal. The proposal left this to the architect agent.
4. **Cycles are merged, not rejected.** Rejecting them left gamma j with nothing to run, since checkout writes an order and the profile page reads it.
5. **A second, simpler L1/L2 path.** To get L2 numbers before this deadline, we added a path where each L0 spec is split into exactly two chained contracts (`Input` → `Mid` → `Output`) and one coder per contract proves PO3. The graph path remains the main design.
6. **Not started yet:** incremental re-verification when requirements change, and the baselines that need it (B4).

4 Demonstration on gamma j

The login requirement shows how the layers work together. L0 produces an `Input` with the entered credentials and a list of accounts, an `Output` with `signedIn`, and `post` stating `signedIn = true` ↔ ∃ `a` ∈ `accounts`, with matching name and password. Lean proves a satisfiability example. L0.5 then changes the output and confirms `post` rejects it, and the negative check proves that "wrong password, signed in" is rejected; the first login spec we generated failed exactly this test. In L1 the spec is assigned to the `Account` component, the model writes `Account_pre_login` and `Account_post_login`, and the harness proves the trace obligation and PO1.

On the simpler path, a use case such as "sales person removes a product" comes out as one Lean file with the spec, two contracts, two implementations and all three proof obligations proved. Every run writes self-contained output: each Lean file compiles from the repository root with `lake env lean <file>`, and `arch.json` and `l1l2.json` record every obligation, the attempts and the errors. Unproved L1 obligations are left in `Arch.lean` as `- NOT PROVED:` comments, so a reader can see exactly what is missing.

5 Results so far

L0. Table 1 shows how the compile rate on gamma j changed as we worked on the prompt and the repair loop. Before version 8 every number comes from a single run. When we ran version 8 three times, its own L0.5 pass count moved between 51 and 56, which was larger than most of the improvements we had been reporting, so from then on we compare versions over three seeds.

Version	Main change	Compiling
no Lean check	first prompt, no repair loop	0/75 (checked by hand)
v1	stricter prompt and templates, Lean 3 syntax banned	30/75
v2	conversational repair loop	55/75
v4	staged repair, harness searches for example values, longer context	65/75
v8	fixes to our own checks, mechanical fixups, repair hints	68/75
v8, 3 seeds	same code	67.3 (67–68)/75
v10.1, 3 seeds	string library, name suggestions, truncation guard	67.0 (66–68)/74

Table 1: L0 on gamma j with Qwen 2.5 7B. In v10.1 the parser drops one scope sentence, so there are 74 entries.

Compiling is a weak bar, so we also report L0.5. Over three seeds, v10.1 passes 61 entries (57–63) against 53.3 (51–56) for v8, with fewer attempts per entry (2.16 against 2.58). Most of that difference comes from a change in scoring, though: v10.1 no longer flags `pre = True` when the requirement has no precondition. Counted the same way, the two versions are about equal (roughly 64 and 61). Compilation did not improve beyond the noise either. We froze L0 at this point because further prompt work was moving numbers within the spread.

L1 (graph path). On gamma j the graph has 184 edges and 9 components in 8 groups; 47% of fields map to

a domain entity. Only one group has more than one compiling spec (`AccountSys`, 48 operations), so that is the system we evaluate. Table 2 compares the graph with the no-graph baseline over three seeds.

`AccountSys`, 48 operations, 3 seeds	with graph	no graph
`Arch.lean` compiles	3/3	1/3
Contracts satisfiable and non-vacuous	100%	33%
Trace obligations proved	99%	33% (0–99)
PO1 proved	100%	33%
PO2 proved	0/3	0/3
Minutes per run	12.2	4.6

Table 2: L1 results. Without the graph, two of three runs produced a decomposition the harness rejected.

With the graph, 96 of 98 obligations are proved in every run. Each run has exactly one failing trace obligation (`3.5.3/S2` twice, `3.1.3/S1` once), and PO2 fails with it because it depends on all traces. The graph runs are clearly more stable, but with three seeds and a baseline that either fails completely or almost succeeds, our comparison tool calls the difference "within noise". We read it as a promising sign rather than a result.

L1 and L2 on the per-spec path. Our first version proved nothing (0/61 on gamma j, 0/14 on video search). Every reply re-declared the locked `Input`, `Output` and `Spec`, the component types often did not chain, and `discharge_po` could not unfold user contracts. After fixing these we got Table 3 (single run, seed 42).

	specs in	L1 proved	L2 components proved	verified end to end
gamma j	61	29/61	48/58	19/61
video search	14	9/14	16/18	7/14

Table 3: Per-spec L1 and L2. "End to end" means L1, every L2 component and the combined file all check in Lean. On gamma j this is 12/27 requirements, 7/17 use cases and 0/17 data constraints. The gamma j run took 65 minutes of model time, split over five GPU jobs.

These numbers overstate the progress. In 12 of the 19 gamma j successes and 3 of the 7 video-search ones, the first component just copies its input to its output, so the proof is valid but the decomposition does not mean much. About seven are real two-step designs. Once L1 holds, L2 usually succeeds, which suggests the hard part of the pipeline is the architecture, not the code.

6 Known failures and limitations

- **A spec that compiles can still be wrong.** L0.5 catches some of this, but one negative example is weak evidence: it finds a `post` that accepts too much, not one that rejects valid outputs. Vague requirements ("confirm orders") still give specs that say too little. We have checked only part of the specs by hand.
- **Persistent L0 failures.** `5.5/R1` (a shortest-path requirement) and `DD-shipping-date` fail in every version; the card expiry date, `DD-location` and the checkout use case fail in about two of three runs. The typical causes are invented library functions, type errors on tuples and date formats. Specs that do not compile are left out of L1, and losing checkout hurts, because it connects the cart and the orders.
- **PO2 is not proved on the graph path.** The failing trace obligation reports "Expected type must not contain free variables", which looks like a problem in how the harness builds the statement, not in the model's contract. We have not looked into it yet.
- **Trivial contracts.** A contract that repeats the L0 conditions makes every obligation easy, and nothing in the pipeline rewards a contract that says more. On the per-spec path this shows up as the copy-only components above.
- **Data constraints cannot be split in two.** All 17 gamma j data constraints fail per-spec L1. A data constraint is a single validity check, and the model moves it into the second component's precondition, which makes PO2 false. These specs should skip L1 or be allowed one component.
- **The graph depends on its inputs.** Only 47% of fields map to an entity, and session state such as `signedIn` has none. Seven of the eight groups are single data fields, so the graph path has been evaluated on one system only.
- **Limited evidence.** One model size (7B), one main SRS, three seeds for L0 and graph L1, one seed for the per-spec runs. The video-search SRS was read while we wrote L0 version 9, so it is not a clean held-out test. We have not analysed the L2 failures yet.

7 Remaining work and plan

1. **By 12 Nov:** find and fix the failing trace obligation so PO2 and `system_sound` go through on `AccountSys`, then run L2 on the graph path (the `code` stage) with three seeds.

2. **By 15 Nov:** a semantic check in L1 that rejects copy-only components and copies of the spec; one-component handling for data constraints; the rule that `Mid` carries every field the second component needs. Rerun the per-spec path with three seeds.
3. **By 19 Nov:** one comparison run with a larger model (Gemma over the API) to separate the 7B model's limits from our pipeline's; a truly held-out SRS from PURE that nobody on the team has read.
4. **By 24 Nov:** incremental re-verification (`harness/reverify.py`): when a requirement changes, use the graph and trace links to recheck only the affected contracts, compared against regenerating everything.
5. **By 30 Nov:** final runs, the 8-page ACM report and the demo. Also on the list: `requirements.txt`, folding the per-spec path into `main.py`, more manual review of generated specs, and revoking an API key that was committed early in the project.

We would especially like feedback on two choices: whether evaluating on real SRS documents instead of the planned hand-made benchmark is acceptable for the final submission, and whether the per-spec L1/L2 path is worth keeping next to the graph path or should be dropped once the graph path works.